Working with Data

Session 1 — What is this data?

Block 1 — Tables, grain, keys, types, and missing values

This block in one sentence

A table is a claim: *one row is one ___*.

Until you can finish that sentence — and prove it with a query — every number you compute from the table is a guess.

Agenda

1. What a table is · **grain** · **keys**
2. The inspection reflex: four queries you run on every table
3. Reading rows: your first `SELECT` — and a filter written once, as a **view**
4. **Missing**: what `NULL` does to a filter, and to a sum
5. Lab 1 — on a different file

Two files, two jobs

Lab 1's project, which you clone in the stretch, carries **two** datasets in `data/raw/`:

File	What it is	Who uses it
`online_retail.parquet`	525,461 invoice lines of a UK online gift shop, Dec 2009 – Dec 2010	**this lecture**
`owid_co2.csv` (+ its codebook, `owid_codebook.csv`)	Our World in Data's CO_2 emissions	**the lab**

The lecture runs on the shop's invoices. The lab's file you meet cold.

In the lecture you predict, with a reason, then watch. You type in the lab. Laptops are for notes.

What a table is

- **Columns are variables.** Each one measures one thing, in one unit.
- **Rows are observations.** Each one is one instance of the thing the table is about.
- **One kind of thing per table.** Invoices here; customers in another table.

That is Hadley Wickham's *tidy data*, in three lines. Real files break every one of them.

Grain: one row is one ___

Table	One row is one…	So the key is…
a list of customers	customer	`customer_id`
invoice lines	product on one invoice	(`Invoice` , `StockCode`)?
country emissions	country in one year	(`country` , `year`)

A **key** is the column, or columns, that identify one row. A key has two properties: **unique for every row**, and **never missing**.

The inspection reflex

Four queries. Every table. Every time. Before any number.

1. `DESCRIBE` — what did DuckDB decide?

Predict, then watch.

```
con.sql("DESCRIBE SELECT * FROM 'data/raw/online_retail.parquet'").df()
```

Predict: what type will DuckDB give `Customer ID`: a whole number, a number with decimals, or text?

1. `DESCRIBE` — what it printed

cell 1

```
con.sql("""
    DESCRIBE SELECT * FROM 'data/raw/online_retail.parquet'
""").df()
```

	column_name	column_type	null	key	default	extra
0	Invoice	VARCHAR	YES	None	None	None
1	StockCode	VARCHAR	YES	None	None	None
2	Description	VARCHAR	YES	None	None	None
3	Quantity	BIGINT	YES	None	None	None
4	InvoiceDate	TIMESTAMP	YES	None	None	None
5	Price	DOUBLE	YES	None	None	None
6	Customer ID	DOUBLE	YES	None	None	None
7	Country	VARCHAR	YES	None	None	None

2. `SUMMARIZE` — every column, one row each

Watch.

cell 2

```
con.sql("""
    SUMMARIZE SELECT * FROM 'data/raw/online_retail.parquet'
""").df()
```

	column_name	column_type	min	max	null_percentage
0	Invoice	VARCHAR	489434	C538164	0.00
1	StockCode	VARCHAR	10002	m	0.00
2	Description	VARCHAR	DOORMAT UNION JACK GUNS AND ROSES	wrong invc	0.56
3	Quantity	BIGINT	-9600	19152	0.00
4	InvoiceDate	TIMESTAMP	2009-12-01 07:45:00	2010-12-09 20:01:00	0.00
5	Price	DOUBLE	-53594.36	25111.09	0.00
6	Customer ID	DOUBLE	12346.0	18287.0	20.54
7	Country	VARCHAR	Australia	West Indies	0.00

7 more columns not shown

Read two columns of the result: **`min`** / **`max`** (is anything impossible?) and **`null_percentage`** (what is missing?).

3. The key test

Predict, then watch. If (`Invoice` , `StockCode`) is the key, the two numbers are equal:

```
con.sql("""
    SELECT COUNT(*)                                  AS n_rows,
           COUNT(DISTINCT (Invoice, StockCode)) AS n_keys
    FROM 'data/raw/online_retail.parquet'
""").df()
```

Predict first: equal, or not?

3. The key test — what it printed

cell 3

```
con.sql("""
    SELECT COUNT(*)                                  AS n_rows,
           COUNT(DISTINCT (Invoice, StockCode)) AS n_keys
    FROM 'data/raw/online_retail.parquet'
""").df()
```

	n_rows	n_keys
0	525461	512126

Not a key

```
n_rows    525,461
n_keys    512,126
```

13,335 more rows than keys. Two reasons, both real:

- the same product entered **twice on one invoice**, as two lines;
- **6,865 lines that are exact copies** of another line, every column identical.

The file cannot tell you whether six identical lines are six sales or one sale recorded six times. **You decide, and you write the decision down.**

4. The census

Predict, then watch. Every value of a column, and how many rows have it — **every** value, never the top few:

```
con.sql("""
    SELECT Country, COUNT(*) AS n
    FROM 'data/raw/online_retail.parquet'
    GROUP BY Country
    ORDER BY n DESC
""").df()
```

Predict: would every value of `Country` match a name in a standard list of countries? Then read all of them, to the end.

4. The census — what it printed

cell 4

```
con.sql("""
    SELECT Country, COUNT(*) AS n
    FROM 'data/raw/online_retail.parquet'
    GROUP BY Country
    ORDER BY n DESC
""").df()
```

	Country	n
0	United Kingdom	485852
1	EIRE	9670
2	Germany	8129
3	France	5772
4	Netherlands	2769
5	Spain	1278
6	Switzerland	1187
7	Portugal	1101
8	Belgium	1054
9	Channel Islands	906

	Country	n
10	Sweden	902
11	Italy	731
12	Australia	654
13	Cyprus	554
14	Austria	537
15	Greece	517
16	United Arab Emirates	432
17	Denmark	428
18	Norway	369
19	Finland	354

4. The census — the other twenty

cell 4 cont.

	Country	n
20	Unspecified	310
21	USA	244
22	Japan	224
23	Poland	194
24	Malta	172
25	Lithuania	154
26	Singapore	117
27	RSA	111
28	Bahrain	107
29	Canada	77

	Country	n
30	Hong Kong	76
31	Thailand	76
32	Israel	74
33	Iceland	71
34	Korea	63
35	Brazil	62
36	West Indies	54
37	Bermuda	34
38	Nigeria	32
39	Lebanon	13

The inspection reflex, in one slide

```
DESCRIBE SELECT * FROM '<file>';                          -- what types?
SUMMARIZE SELECT * FROM '<file>';                         -- what is missing, what is impossible?
SELECT COUNT(*), COUNT(DISTINCT (<key>)) FROM '<file>'; -- is the key a key?
SELECT <col>, COUNT(*) AS n FROM '<file>' GROUP BY <col> ORDER BY n DESC;  -- the census, read to the end
SELECT COUNT(*), COUNT(<col>) FROM '<file>';            -- rows; rows where <col> has a value
```

DS1's four-line pandas check — `shape`, `dtypes`, `head`, `isna` — spelled in SQL. Also on `REFERENCE-sql.md` in the lab project.

Reading rows: your first `SELECT`

`SELECT` · `FROM` · `WHERE` · `ORDER BY` · `LIMIT`

Predict, then watch. The ten largest quantities ordered from Japan:

```
con.sql("""
    SELECT Invoice, Description, Quantity, Price
    FROM 'data/raw/online_retail.parquet'
    WHERE Country = 'Japan'
    ORDER BY Quantity DESC
    LIMIT 10
""").df()
```

Say it as a sentence first: *which columns, from where, which rows, in what order, how many*. Predict: with `"Japan"` in double quotes instead of `'Japan'`, the same ten rows, or an error?

`SELECT` · `FROM` · `WHERE` · `ORDER BY` · `LIMIT` — what it printed

cell 5a

```
try:
    con.sql("""
    SELECT Invoice, Description, Quantity, Price
    FROM 'data/raw/online_retail.parquet'
    WHERE Country = "Japan"
    ORDER BY Quantity DESC
    LIMIT 10
""").df()
except duckdb.Error as e:   # the error is caught so the notebook keeps running; its first line is the point
    print('With "Japan" in double quotes, DuckDB refused:', str(e).splitlines()[0])
```

```
With "Japan" in double quotes, DuckDB refused: Binder Error: Referenced column
"Japan" not found in FROM clause!
```

The query as written, with `'Japan'`

cell 5b

```
con.sql("""
    SELECT Invoice, Description, Quantity, Price
    FROM 'data/raw/online_retail.parquet'
    WHERE Country = 'Japan'
    ORDER BY Quantity DESC
    LIMIT 10
""").df()
```

	Invoice	Description	Quantity	Price
0	537899	ROUND SNACK BOXES SET OF 4 FRUITS	1488	2.55
1	527041	RAIN PONCHO RETROSPOT	432	0.65
2	527041	PACK OF 12 TRADITIONAL CRAYONS	288	0.36
3	527041	PACK OF 12 COLOURED PENCILS	288	0.72
4	527041	SET 12 KIDS COLOUR CHALK STICKS	288	0.36
5	527041	SET 12 KIDS WHITE CHALK STICKS	288	0.36
6	527041	POLKADOT RAIN HAT	288	0.32
7	527041	SEWING SUSAN 21 NEEDLE SET	288	0.64
8	527041	MAGIC DRAWING SLATE GO TO THE FAIR	192	0.36
9	527041	SNOWSTORM PHOTO FRAME FRIDGE MAGNET	192	0.72

Rows that are not the thing

Watch. The stock codes that are not products:

cell 6

```
con.sql("""
    SELECT DISTINCT StockCode, Description
    FROM 'data/raw/online_retail.parquet'
    WHERE StockCode IN ('POST', 'DOT', 'C2', 'BANK CHARGES')
    ORDER BY StockCode
""").df()
```

	StockCode	Description
0	BANK CHARGES	Bank Charges
1	BANK CHARGES	Bank Charges
2	C2	NaN
3	C2	CARRIAGE
4	DOT	NaN
5	DOT	DOTCOM POSTAGE
6	POST	NaN
7	POST	POSTAGE

Postage, carriage, bank charges — on the same rows, in the same columns, as the products. `SUM(Price)` adds them all.

And `Bank Charges` appears **twice**: one spelling starts with a space. The census would count them as two values.

The fix, written once: a view

Watch (cell 7). A **view** is a saved query with a name. The filter is written once; every later query reads `FROM products` instead of the file.

cell 7

```
con.sql("""
    CREATE OR REPLACE VIEW products AS
    SELECT *
    FROM 'data/raw/online_retail.parquet'
    WHERE StockCode NOT IN ('POST', 'DOT', 'C2', 'BANK CHARGES')
""")

con.sql("SELECT COUNT(*) AS product_lines FROM products").df()
```

	product_lines
0	523657

`CREATE OR REPLACE`: make it, or remake it if it exists, so the cell can run again. The lab asks you for one.

Missing

"Missing" has many spellings

In the file	Means
an empty cell → `NULL`	we do not know
`N/A` , `:` , `-999` , `UNKNOWN`	we do not know — written as a value
`0`	maybe zero, maybe *not measured*
a row that is not the thing	the value belongs to something else

`NULL` is SQL's one word for *unknown*. Everything else you have to find — with the census.

`NULL` in a `WHERE`

Predict, then watch (cell 8) — three queries, three counts, one number kept from each:

```
manual = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description = 'Manual'
""").fetchone()[0]
not_man = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description != 'Manual'
""").fetchone()[0]
missing = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description IS NULL
""").fetchone()[0]
print(manual, not_man, missing, "of", manual + not_man + missing)
```

`.fetchone()[0]`: the first row of the result, then its first column — the number itself, not a table. Predict: do the first two add up to 525,461, the file's row count from the key test?

`NULL` in a `WHERE` — what it printed

cell 8

```
manual = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description = 'Manual'
""").fetchone()[0]
not_man = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description != 'Manual'
""").fetchone()[0]
missing = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description IS NULL
""").fetchone()[0]
print(manual, not_man, missing, "of", manual + not_man + missing)
```

```
854 521679 2928 of 525461
```

854 + 521,679 is not 525,461. **A comparison with a missing value is neither true nor false: it is *unknown*.** **`WHERE` keeps only the rows where the test came out true.** So the 2,928 rows with no description fail both tests, and appear in neither count.

Counting the missing with `= NULL`

Predict, then watch (cell 9). 2,928 rows have no description. What does this count?

```
con.sql("SELECT COUNT(*) FROM 'data/raw/online_retail.parquet' WHERE Description = NULL").df()
```

Counting the missing with `= NULL` — what it printed

cell 9

```
con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet' WHERE Description = NULL
""").df()
```

	count_star()
0	0

`= NULL` **is never true**, so the query returns nothing, and no error. The test for missing is `IS NULL`.

`IS NOT NULL`, and `OR`

Watch (cell 10). The rows that *have* a description; and "everything except Manual" written so the rows with no description are kept:

cell 10

```
has_desc = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description IS NOT NULL
""").fetchone()[0]

not_man = con.sql("""
    SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'
    WHERE Description != 'Manual' OR Description IS NULL
""").fetchone()[0]

print("has a description:", has_desc, "  everything except Manual, NULLs kept:", not_man)
```

```
has a description: 522533   everything except Manual, NULLs kept: 524607
```

On screen, pandas prints a `NULL` as `NaN` or `None`. **Neither word is in the file.** The only tests are `IS NULL` and `IS NOT NULL`.

Plus poisons? `SUM` skips?

Predict, then watch (cell 11). Three real cells of Eurostat's municipal waste file: Austria, kg per inhabitant. The file writes 2024 as `:`, *not available*: here, `NULL`. The three rows are typed in as a tiny view, `austria`, so the query reads a table like any other.

```
con.sql("""
    CREATE OR REPLACE VIEW austria AS
    SELECT * FROM (VALUES (2022, 803), (2023, 782), (2024, NULL)) AS t(yr, kg)
""")

con.sql("""
    SELECT 803 + 782 + NULL AS with_plus,
           SUM(kg)          AS with_sum,
           AVG(kg)          AS average
    FROM austria
""").df()
```

Predict: `with_plus`, `with_sum`, and `average`.

Plus poisons? `SUM` skips? — what it printed

cell 11

```
con.sql("""
    CREATE OR REPLACE VIEW austria AS
    SELECT * FROM (VALUES (2022, 803), (2023, 782), (2024, NULL)) AS t(yr, kg)
""")

con.sql("""
    SELECT 803 + 782 + NULL AS with_plus,
           SUM(kg)          AS with_sum,
           AVG(kg)          AS average
    FROM austria
""").df()
```

	with_plus	with_sum	average
0	<NA>	1585.0	792.5

Plus poisons; `SUM` skips

`803 + 782 + NULL`	`NULL` : arithmetic with a missing value has no value
`SUM(kg)`	1,585: aggregates **skip** `NULL`
`AVG(kg)`	792.5: 1,585 ÷ 2, not ÷ 3. The missing year is left out, not counted as a zero

What you just saw, as one idea

A table can contain rows that are not the thing one row claims to be — a copy, a postage charge, a country called *Unspecified*, a whole region, the world.

- `SUM` does not care. `COUNT(*)` does not care. **No error.**
- The key test and **the census, read to the end,** are how you find them.
- The fix is never "delete the row I noticed." It is **a filter that says what the thing is** — and a check that proves the filter.

Git and AI, this block

Git: commit after each fix, and the message names the cause.
`Exclude postage and bank charges from product revenue: they are not products` — not `fix`.

AI: ask an assistant which rows a dataset usually contains. Then run the census.
*The assistant knows what a dataset usually looks like. The census tells you what **this** file contains.*

The diagnosis note, for data

1. **Symptom** — which number was wrong, and what did you compare it to?
2. **Cause** — why the old query produced that number (not what you changed).
3. **Evidence** — *paste* the queries you ran and what they returned.
4. **Change** — *paste* the query as it is now.
5. **Verification** — *paste* the check that passes now, and the two numbers that now agree: the **identity**, two routes to one number that must tie out.

Paste the evidence **before** you change anything. A fix erases the output that proved the cause.

5-minute stand-and-stretch — start the clone first

In your terminal (Git Bash on Windows, Terminal on macOS), in your course folder, **then take your break**:

```
git clone https://github.com/earino/ecbs5294-lab01-grain.git
cd ecbs5294-lab01-grain
uv sync
```

It downloads while you stretch.

Lab 1 — The planet that emits six times too much

The file: `data/raw/owid_co2.csv` — Our World in Data, every country, every year. The codebook: `data/raw/owid_codebook.csv`.

The symptom: the notebook's report runs without an error and says the world emitted about **245,700 Mt** of CO_2 in 2024, China's share was **5%**, and **254** countries reported. The file has its own figure for the whole world — a row called `World` — and it disagrees.

Your work, in the notebook, top to bottom. **A:** run the four inspection recipes on the file and read what they return. **B:** write the filter as a view, `countries`, and its opposite, `excluded`; read what you excluded; a supplied check proves the two views split the file. **C:** the ten largest emitting countries, from an empty cell. **D:** a supplied check puts the `World` row beside the sum over your view; what it leaves over must be rounding, and if it is not, your filter is wrong. **E:** the corrected report. Then `DIAGNOSIS.md`. Neighbour at minute 33. Checkpoint on Moodle. Syntax: `REFERENCE-sql.md` in the project; the full reference is on the course site.

The last ten minutes — explain it to a neighbour

One of you explains, about a minute, pointing at the screen: **what was wrong · why · the query that proved it · what you changed · how you know it is right.** The other asks:

1. **Show me the query that proves it.**
2. **Why was it wrong, not just where?**
3. **Run your check for 1990 instead of 2024. The row your filter names has no value at all for 1990. Does the check still close?**

Then swap. Unsure, or you disagree? **Hands up.** Then the checkpoint on Moodle.